Performance and Functional Testing

Phase 6: Project Testing

1. Testing Purpose

Testing checks that the deployed application behaves as expected and that failures are visible to the user. This document describes the test method and expected outcomes. No benchmark logs or executed test-run results were supplied, so no measured performance is claimed and every status below remains "Pending verification" until it is observed.

2. Environment and Test Data

Use the deployed frontend at https://travel-book-livid.vercel.app, or a local setup as described in the run instructions. A representative test case: Kyoto, Japan; 10 to 13 November 2026 (4 days); 2 travellers (couple); budget 3,200 USD; balanced style; moderate pace; interests Culture and Heritage, Food and Culinary, Sightseeing; dietary note "Pescatarian, loves matcha and ramen". Confirm the dates and values before recording a demonstration.

3. Automated Tests in the Repository

Suite	File	Tests	What they cover
Backend	test_health.py	2	/health and root endpoint responses
Backend	test_validation.py	5	Valid request, end date before start date, budget too low, zero travellers, invalid time slot
Backend	test_ai_validation.py	2	Itinerary structure; single-day regeneration
Backend	test_budget.py	2	Within-budget and over-budget breakdown
Backend	test_trips_api.py	6	List, get, missing trip, create and delete, patch metadata, HTML export
Frontend	formatters.test.js	4	USD and EUR formatting, inclusive day count, duration formatting
Frontend	mockData.test.js	2	Sample trips are valid; client itineraries are customized

The repository README reports all 17 backend and 6 frontend tests as passing. These tests were not executed while preparing this documentation, so the result should be reproduced with pytest and npm run test and recorded in the results log.

4. Functional Test Areas

Deployment smoke test

Open the frontend and confirm the landing page loads and routes such as /trips load on refresh. If the backend is deployed, request its /health endpoint and record the response and time.

Authentication

Use the one-click demo sign-in and confirm the dashboard opens with the demo banner. Test sign-up with a password shorter than six characters and confirm that a validation message appears. Sign out and confirm that /dashboard redirects to /login.

Wizard and generation

Leave the destination blank, enter an end date before the start date, set a budget below 100 and select no interest; confirm each is blocked. Then submit the representative case and confirm that a loading state appears and the itinerary page opens with day tabs and a budget summary.

Budget

Set a very low budget and confirm an over-budget warning; set a very high budget and confirm the remaining-budget message. Edit an activity cost and confirm the total changes.

Saved trips

Search by destination, filter by style, open a trip, edit it and delete a disposable trip after confirming the dialog. Reload and confirm the list is consistent.

Print and save as PDF

Open Print / PDF and inspect the preview: navigation, sidebar and buttons are hidden, all days are present and no section is clipped.

Not-found page

Open a non-existent address and confirm the custom 404 page appears.

5. Basic Performance Measurement Method

Record the start and completion time of several generations, separating backend cold-start delay from provider response time where possible. Repeat with the same input and record success or failure. Do not report averages until enough real observations exist.

6. Error and Reliability Checks

- Disable the AI key in a controlled environment and confirm the fallback generator still returns a plan.
- Open the site with the backend unreachable and confirm the frontend falls back to demo data.
- Request an unknown trip identifier and confirm a 404 response.
- Send a request with an invalid date range directly to the API and confirm a validation error.
- Confirm the interface stays usable after a failed request.

7. Results Log Template

Date/time	Environment	Input	Expected	Actual	Pass/Fail	Evidence file
					Pending verification	

Mark a test as passed only after observing the expected behaviour.